The Residual of Logic

Revisability as a Condition of Continuity in Self-Describing Knowledge Systems

Mitchell D. McPhetridge Independent Researcher Revised 11 November 2026

Abstract

A persistent knowledge system may preserve a conclusion while losing the constraints that could reject its present authority. This paper calls that condition residual failure and distinguishes persistence, recoverability, and enforced revisability. The McPhetridge 2024–2026 research arc supplies a documented self-application case: an account claiming completion is subsequently subjected to the same eliminative requirements applied elsewhere in the corpus, and its closure claims are pruned. Combined MPLPB supplies a separate executable case of sealed identity, pinned lineage, retirement, ownership ambiguity, and refusal. An exploratory pilot executes fourteen synthetic fixtures against four reading conditions: first-file selection without a gate, newest-version selection without a gate, MPLPB's validated interfaces, and an independently coded conventional reference gate. MPLPB and the reference gate return identical normalized decisions on this suite: no ineligible pages served in eleven opportunities, one ownership conflict detected, both stale derivatives rejected, no false support promotion in eight unanswerable cases, and six of six answerable cases covered. These small, purpose-built measurements demonstrate bounded enforcement behavior. Their equivalence also limits claims of architectural necessity or superiority. The contribution is an operational criterion for continuity: disqualification must remain available and effective alongside what is remembered. Neither ledger eligibility nor successful retrieval establishes factual truth.

Keywords: continuity; revisability; provenance; supersession; refusal; self-application; knowledge systems; MPLPB

1. Introduction: Recovery Is Not Continuation

Suppose a knowledge system resumes after its originating author, model session, or runtime disappears. A later reader recovers an index, earlier conclusions, relevant evidence, and a next action. State has survived interruption. But a second question remains: what can that reader still reject?

A recovered claim may have been superseded. A derivative may depend on a retired basis. Two pages may compete for the same scope. An intact source may be relevant to a question without supporting the requested answer. In each case, faithful recovery of the bytes can coexist with an invalid continuation.

This paper distinguishes three properties. Persistence means that the record survives. Recoverability means that a later reader can locate and interpret it. Residual enforcement means that the reader's permitted continuation remains constrained by the record's current

eligibility, lineage, provenance, ownership, and support requirements. The third property does not follow from the first two.

The thesis is limited accordingly: continuation is trustworthy only with respect to constraints that remain available and are actually applied after recovery. A system need not decide every proposition. It must retain a way to withhold authority where its declared conditions fail.

Two kinds of evidence develop this argument. The Arc and Critical Self View papers document application of a method to its writer's own claims [1, 2]. An executed engineering pilot tests whether specified software paths enforce related eligibility and refusal rules [3, 4]. The historical correction and the software experiment support different propositions; neither substitutes for the other.

2. The Research Arc: The Method Corrects Its Writer

The self-application case is attributable. In This Arc: 2024 — Late May 2026 and Tomorrow, McPhetridge presents the development from early generative expansion through compression, constraint, and externalization as having reached completion and self-regulation [1]. The public account treats the resulting architecture as able to continue beyond its builder.

Critical Self View (reflective/meta layer) Arc 2024–2026 Pruning then makes that account itself a candidate for evaluation [2]. Its indexed primary-source text identifies the completion narrative as an instance of inward recursion: the account integrates its development into a closure claim without supplying further eliminative tests for that closure. The critique applies the corpus's own requirements, including DRE, the persistence condition, the Straight-Line Boundary, and the distinction between inward and outward recursion. It rejects the closure elements while retaining the useful structural documentation.

The direction of correction matters. The writer does not reserve an exemption for the writer's own culminating interpretation. A method initially used to evaluate other structures is applied to the claim that the method's development is complete. The critique states the consequence directly: “The machine runs on the builder too.” [2]

This is a documented substantive self-application case, rather than an anonymous hypothetical. It shows the method being used to correct its writer's account. It does not establish that an autonomous software process independently generated the criticism, that every reader accepts the judgment, or that a current ledger already encodes the precise retirement of those historical claims. The available record supports public correction under stated standards; automated enforcement of that particular correction requires additional provenance and execution evidence.

The research arc concerns 2024–2026. The archive records consulted list archival dates of 18 May 2026 for the Arc paper and 20 May 2026 for the pruning paper [1, 2]. Those metadata dates should not be confused with dates inferred from titles or later descriptions. In this review, direct record downloads were blocked by HTTP 403; the historical analysis

uses accessible search-indexed primary-source record text and version metadata. Complete original source files and historical MPLPB seals were not obtained.

The pair illustrates the paper's central failure condition. If a later reader recovers only the completion account, or retrieves both papers without preserving their evaluative relationship, the record may survive while its correction loses effect. More generally, a conclusion can outlive the test that once constrained it. A continuity architecture must preserve the means of disqualification as well as the conclusion.

3. An Operational Definition of the Residual

Let a corpus contain records with identities, scopes, status declarations, content seals, origins, and version or dependency relationships. Let a read occur under a declared question and delivery profile. A residual is a continuing constraint mechanism capable of preventing a record from being treated as presently authoritative when its eligibility or support conditions fail.

The definition distinguishes four decisions.

Identity and integrity. The record being used must be the record identified by the corpus, and its content must match the recorded seal. A matching SHA-256 digest demonstrates consistency with a reference digest. It does not authenticate authorship, establish factual correctness, or prevent a privileged operator from replacing both content and digest.

Status and lineage. A retired predecessor remains inspectable as history. A valid successor controls current eligibility under the declared replacement rules. An invalid successor cannot acquire retirement authority merely by naming a predecessor. A derivative whose basis is retired must be withheld until an explicit recheck establishes a current basis.

Ownership and conflict. A question and scope may select no owner, one owner, or competing eligible owners. Where the declared rules leave genuine competition, the interface must expose ambiguity or withhold an answer. An arbitrary ranking must not silently become a claim of unique authority.

Answer support. Finding an eligible page differs from establishing that its body supports the requested proposition. Candidate inspection, gated page retrieval, and supported answering have different contracts. An interface can successfully retrieve a source while refusing a supported answer.

These requirements define a negative capability: the system preserves the ability to reject a claim's present use on inspectable grounds. They do not constitute a general logical calculus or a truth-verification algorithm. The effective property exists at interfaces that apply the checks. A raw-file consumer can bypass a guarded interface, and a written rule alone does not prevent that bypass.

4. Combined MPLPB as an Executable Case

Combined MPLPB is a local ledger of self-describing pages with provenance checks, version and dependency relationships, and explicit read decisions [4]. The implementation examined here is version 2.1.0rc1 at commit `257e54a51edac272eb6b2441457f38a40485ee4d`. Pinning the commit matters because a description of a changing repository is not a stable experimental target.

The supersession policy separates replacement authority from file presence [5]. A valid replacement can retire a predecessor. A damaged or incorrectly pinned replacement cannot do so by declaration alone. Withdrawing a replacement does not automatically reactivate an older page. A derivative of a retired basis is withheld until explicitly rechecked. The policies preserve history while restricting current use.

The read interfaces also separate eligibility from source support [6].

Interface	Successful operation	Limit
Candidate inspection	Identifies lexical ownership candidates	Does not establish body support
Gated retrieval	Returns one selected source after provenance checks	Sets `answer_supported=false`
Strict answering	Returns a supported statement under bounded source-language rules	Does not establish source truth or general language understanding
Validation	Checks recorded page and ledger invariants	Does not prove every claim in a valid page

The pinned CLI exposes candidate inspection and strict answering. Gated retrieval is available as the Python function `mplpb_combined.retrieval.retrieve`; the experiment invokes it through a small adapter. There is no `retrieve` CLI subcommand at this commit. This distinction is retained in the reproduction instructions rather than hidden behind an invented command.

The upstream ownership evaluation contains synthetic developer-authored cases [7]. Those archived results remain engineering context. The measurements reported below come from the new, separately retained residual pilot.

5. Executed Pilot: Design and Conditions

5.1 Scope and evidence retention

The exploratory pilot uses fourteen isolated fixtures, four reading conditions, and two read interfaces per condition. It executes 112 fresh reader processes, fourteen fixture-validation processes, and six additional public-corpus operations. The recorded Python version is 3.12.14. The upstream tracked source was clean and was not modified.

Fixture construction uses the pinned MPLPB writer API. Every reader reconstructs its state in a fresh subprocess. This tests process-level recovery from persisted corpus files; it does not simulate a different machine, a new human evaluator, or loss of the original software. Labels and fixture digests are saved before any read condition runs. Expected eligibility and support labels are explicitly specified in the runner rather than copied from MPLPB's read decisions.

The study was neither preregistered nor blind. Its fixtures and baseline were developed with knowledge of the mechanism being tested. The outputs are therefore a bounded engineering pilot, not independent certification or a representative estimate of deployment performance.

The repository retains commands, working directories, exit codes, stdout, stderr, Python and source versions, script hashes, page seals, and fixture file digests [3]. Deliberate corruptions produce validation failures; those failures are retained as observations. An evidence check verifies fixture immutability, selected read contracts, the public chain, and agreement between conditions C and D.

5.2 Fixtures

The fourteen fixtures cover an intact supported page; valid revision; a tampered page; a tampered successor; two competing eligible owners; a derivative after basis replacement; a derivative after basis withdrawal; a withdrawn successor chain; a matching-scope page without body support; an absent owner; a wrong supersession pin; duplicate identity; an explicitly rechecked derivative; and revision under the same declared owner.

Queries use the bounded form asking for the color of a named object. Supported sources make direct affirmative color statements. This small grammar lets the experiment isolate eligibility and refusal without treating arbitrary prose interpretation as a solved problem. The unsupported-body control assigns the queried scope to a page whose body concerns another object.

The tampered-successor and wrong-pin cases explicitly restore the predecessor's declared status to current. They test whether an invalid successor can retire an otherwise current predecessor, following the published incomplete-revision setup [5]. They do not assume that damaging a successor reverses an independently recorded withdrawal. The same-owner fixture is synthetic and mechanically resembles valid revision; it is not a reconstruction of the historical Arc and adds no independent evidence about that history.

5.3 Four reading conditions

A — first-file selection without enforcement. A reader performs exact declared-scope lookup and selects the first matching file in sorted path order. It applies no eligibility gate. Its constructed answer consumer promotes the selected prose to an answer without a body-support test.

B — newest-version selection without enforcement. A reader has the same files and their version, status, and lineage metadata, and selects by descending timestamp and identifier. It applies no enforced eligibility gate. Its constructed answer consumer uses the

same unchecked promotion behavior as A.

C — MPLPB enforcement. The reader invokes provenance-gated retrieval and the strict supported-answer CLI. Retrieval never claims support merely because it returns a page. Strict answering can return, report ambiguity, or refuse.

D — conventional reference gate. A separately coded reader uses standard-library HTML parsing, the published seal serialization, unique identity checks, pinned lineage, status and withdrawal checks, dependency freshness, exact scope ownership, and an affirmative color grammar. It imports no MPLPB enforcement code. More than one eligible owner causes ambiguity; missing support causes refusal.

All conditions receive the same corpus bytes and frozen questions. Conditions A and B are specified ablations, not empirical claims about all file readers, Git workflows, or databases. Their answer-promotion errors belong to the constructed consumers, not to the storage format itself. D supplies the stronger comparison missing from a simple guarded-versus-unguarded contrast: a conventional mechanism explicitly configured to stop and refuse.

D is a narrow reference implementation rather than a production alternative. It shares the published serialization contract with MPLPB and was developed against this suite. It does not implement the complete query language, delivery-profile system, ratification policies, or compatibility formats. Agreement on these fixtures cannot establish general equivalence, usability parity, or equal maintenance cost.

5.4 Five metrics

The ineligible-claim serving rate counts retrieval of a labelled ineligible identifier among eleven query fixtures containing an ineligible source opportunity. This is a per-query opportunity rate, not the fraction of served pages that are ineligible. Ambiguous-ownership detection counts an explicit ambiguity decision on the one competing-owner fixture. Stale-derivative rejection counts withholding a source on the two stale-derivative fixtures.

Supported-answer false promotion counts `answer_supported=true` among eight fixtures without a unique eligible supported answer. This includes eligibility failures and ambiguity, as well as the body-support-negative control. Answerable-query coverage counts a supported answer from the labelled eligible supporting identifier among six answerable fixtures. These denominators overlap across metrics; they are not disjoint populations.

6. Results and Interpretation

6.1 Measured pilot results

Metric	Preferred direction	A	B	C	D
Ineligible-claim serving	Lower	9/11	7/11	0/11	0/11
Ambiguous-ownership detection	Higher	0/1	0/1	1/1	1/1
Stale-derivative rejection	Higher	0/2	0/2	2/2	2/2
Supported-answer false promotion	Lower	7/8	7/8	0/8	0/8
Answerable-query coverage	Higher	3/6	4/6	6/6	6/6

MPLPB and the conventional reference gate produce identical normalized retrieval and answer decisions across all fourteen fixtures. Each avoids the labelled eligibility and promotion failures while answering all six answerable queries. This prevents a trivial always-refuse policy from explaining their success on this suite.

The results also limit the engineering claim. A conventional gate can provide the same observed protection on this subset. The pilot therefore does not show that the residual requires MPLPB's particular architecture. It demonstrates an enforceable criterion and two implementations satisfying that criterion under the tested conditions.

The difference between A and B shows a narrower point: preferring a newer file can avoid some obsolete selections, but recency alone does not enforce seal integrity, valid retirement authority, dependency freshness, unique ownership, or answer support. Neither ablation is a competitive estimate of a fully configured conventional system; D is included precisely because conventional systems can enforce such rules.

6.2 Retrieval without support

The unsupported-body fixture isolates the distinction between eligibility and answering. A valid current page, `DOC-0002`, declares the scope of the queried object but supplies a color statement about another object. Both C and D retrieve the page and set `answer_supported=false`. Their answer paths refuse to promote it to a supported answer.

This is not a failure to recover the source. It is successful recovery combined with a correct limit on the operation's authority. A system that conflates retrieval success with answer support would erase that distinction even while preserving every page and seal.

6.3 Falsifiers and what remains untested

The operational claim would fail on a tested interface if it served a labelled ineligible record, silently selected a unique owner from genuine competition, allowed a stale derivative without recheck, or promoted candidate-only retrieval into asserted support. Coverage must accompany refusal measures because blanket abstention can avoid unsafe answers without providing useful continuation.

The present counts are too small and deliberately constructed to establish population error rates. One ambiguity case is a demonstration, not an estimate of robust ambiguity performance. The same-owner and revision controls are correlated. Broader independently authored probes, adversarial metadata variations, delivery restrictions, and held-out language remain necessary.

7. Worked Public-Corpus Self-Reference

The public `examples/system` corpus supplies an executable self-description example distinct from the historical Arc correction. The experiment copies the complete corpus without changing its bytes and runs validation, history, gated retrieval, and strict answering from fresh processes.

Validation reports eight records: one current, seven retired, zero errors, and zero warnings. History retains the chain from `SELF-0001` through `SELF-0008`. The final two pages carry these complete recorded seals:

```
SELF-0007 — retired
sha256:cc1f259ae9c287242350bb236a4700a567a6d8d47775813740976677b9ee7b6f

SELF-0008 — current
sha256:788e1d1979a6137a504ec80301956f03940b8e97cbc185b55f2d5daa6db1675e
```

`SELF-0008` explicitly pins `SELF-0007` in its supersession metadata. All eight recorded seals match the implementation's recomputation; separate full-file digests are retained because a content seal and a file digest cover different representations.

The following commands reproduce the main public observations from the pinned upstream checkout:

```
python3 -m mplpb_combined validate examples/system
python3 -m mplpb_combined history examples/system SELF-0001
python3 -m mplpb_answers ask examples/system 'What is MPLPB?' --answer-mode strict --json
```

The retained validation output ends with:

```
8 record(s): 1 current, 7 retired; 0 error(s), 0 warning(s)
```

The original output includes the corpus's absolute path between `record(s)` and the status counts. It is omitted here for readability; the full unedited output is in `commands.jsonl`. Strict answering returns these fields:

```
{
  "kind": "return",
  "id": "SELF-0008",
  "answer_supported": true,
```

```
  "text": "MPLPB is a closed local ledger of sealed HTML pages."
}
```

Gated retrieval for the system-creator query also selects `SELF-0008` while preserving `answer_supported=false`. The full commands and outputs, all eight seals, origin declarations, effective statuses, and pinned references appear in the evidence directory [3].

These pages declare machine origin and a creator attribution to `mitchell-d00`. That is neither authenticated human authorship nor proof that the software independently corrected its creator. The worked example establishes that the system's own technical description can be revised while earlier descriptions remain historical. The Arc pair establishes substantive application of the method to the writer's closure claims. Encoding and replaying that exact historical correction as a verified original ledger chain remains separate work; archive identifiers `MCPTAD-2` and `MCPCSV` must not be presented as MPLPB page identifiers.

8. Relation to Prior Work and Novelty Boundary

The relevant mechanism families predate this implementation. Version control preserves changes. Database constraints enforce integrity and uniqueness. Truth-maintenance systems represent dependencies among beliefs. Provenance standards describe derivation. Software contracts distinguish the guarantees of one operation from those of another [8–12]. A well-configured system using these established tools can preserve the right to disqualify a record.

The upstream originality reassessment similarly distinguishes established logical principles from unresolved claims about the particular integration [13]. This paper adopts that narrower boundary. Residual logic names a continuity requirement: the means of disqualification must survive and remain operative alongside remembered content. MPLPB is one inspectable implementation case, not evidence that the principle was first invented here.

Condition D strengthens that boundary with an executed comparison. It matches the observed stop/refuse behavior through conventional checks. Distinctive advantages, if any, would need to be demonstrated in implementation scope, policy expression, deployment, auditability, usability, or maintenance under a fair comparison. Those properties were not measured in this pilot.

The Arc case contributes a different kind of evidence. It records an application of the method to its writer's own completion claim. That matters to the continuity argument because a system can accumulate critical language while exempting its organizing narrative from criticism. The correction pair documents the exemption being rejected. It does not establish worldwide novelty, correctness of every surviving claim, or automated author-independent judgment.

9. Limitations and Threat Model

The enforcement boundary is explicit. A reader can open raw files and ignore the gate. A privileged operator can replace content and seals together. SHA-256 consistency is not a signature, an origin field is not identity authentication, and current ownership is not factual truth. Status changes and audit logs remain governed by the implementation's declared local trust model.

The pilot tests process-level reads over constructed local corpora. It does not demonstrate transfer across arbitrary software versions, infrastructure failure, long-term operation, or performance under representative user workloads. The small color grammar deliberately limits semantic variation. A refusal can reflect unsupported evidence or grammar limits rather than the falsity of a proposition.

The baseline is independently coded but not independently evaluated. It shares the published hash contract and was designed with the fixtures in view. Equal results on this subset establish neither complete implementation equivalence nor comparable effort in a realistic application. The study reports no latency, maintenance, usability, or scaling measurements.

Historical evidence also has a boundary. The Arc pair is accessible through indexed public records, while direct downloads were blocked during this review. No original historical ledger seals, status commands, or autonomous evaluation trace for those papers were verified. The documented correction should be cited as such; it should not be upgraded to measured automated reasoning.

Finally, the paper tests enforceable eligibility and source-support contracts, not the substantive correctness of the writer's closure judgment. A later process can apply retirement and dependency rules without independently determining whether the underlying criticism is sound. That deeper evaluation requires explicit criteria and separate measurements.

10. Conclusion

Persistence answers whether a record survived. Recoverability answers whether a later reader can find it. Enforced revisability asks whether that reader can still withhold the record's present authority when the relevant conditions fail.

The Arc and Critical Self View papers document the method being applied to its writer's own account of completion. The public self-reference corpus supplies actual identifiers, seals, history, and executed read outputs for technical revision. The synthetic pilot measures bounded enforcement behavior and shows that a conventional reference gate can match it.

The resulting claim is precise: a continuity architecture preserves the residual only where the checks that allow correction remain operative. Those checks do not confer truth, and their presence in prose is not enough. They preserve the practical possibility of disqualification after the originating author or runtime is absent.

Reproduction and Data Availability

The experiment, baseline, labels, measurements, full command logs, and this paper are maintained at:

https://github.com/mitchell-d00/mplpb-residual-experiment

The retained execution is `results/pilot-1/`. `environment.json` records the source commit, interpreter, script hashes, and label digest. `fixture-digests.json` records input-file digests. `decisions.json` retains normalized decisions, `summary.json` the five metrics, `commands.jsonl` the complete captured processes, and `public-records.json` the public self-reference identities and seals. `verification.txt` records the evidence-check outcome. The runner does not overwrite an existing populated evidence directory.

From a checkout of the experiment repository with the pinned upstream checkout at `../combined-mplpb`:

```
python3 run.py --source ../combined-mplpb --out results/reproduction
python3 check.py results/reproduction
```

The runner uses Python's standard library; no model service is called. The PDF build script separately uses ReportLab and Pandoc. Absolute paths, process times, and generated activity-log timestamps can differ across reproductions; compare normalized decisions, labels, and page seals rather than demanding byte-identical full logs. Original `pilot-1` evidence is retained unchanged.

References

[1] McPhetridge, M. D. This Arc: 2024 — Late May 2026 and Tomorrow (Research Arc of McPhetridge Stack). PhilArchive record MCPTAD-2. Archival date listed as 18 May 2026. Indexed primary-source record consulted 11 November 2026. https://philarchive.org/rec/MCPTAD-2

[2] McPhetridge, M. D. Critical Self View (reflective/meta layer) Arc 2024–2026 Pruning. PhilArchive record MCPCSV. Archival date listed as 20 May 2026. Indexed primary-source record and version metadata consulted 11 November 2026. https://philarchive.org/rec/MCPCSV

[3] McPhetridge, M. D. MPLPB Residual Experiment. Executed exploratory pilot, source, and retained evidence, 11 November 2026. https://github.com/mitchell-d00/mplpb-residual-experiment

[4] McPhetridge, M. D. Combined MPLPB. Version 2.1.0rc1; commit 257e54a51edac272eb6b2441457f38a40485ee4d. https://github.com/mitchell-d00/Combined-MPLPB-A-closed-local-ledger-of-pages-that-answers-only-from-those-pages/tree/257e54a51edac272eb6b2441457f38a40485ee4d

[5] McPhetridge, M. D. Supersession policy examples. `docs/SUPERSESSION_POLICY.md` at the commit in [4].

[6] McPhetridge, M. D. Page retrieval and answer authority. `docs/PAGE_RETRIEVAL.md`, with `mplpb_combined/retrieval.py` and `mplpb_answers/__main__.py`, at the commit in [4].

[7] McPhetridge, M. D. Adversarial semantic ownership evaluation. `evaluation/ownership/README.md` at the commit in [4]. Developer-authored synthetic evidence; not an independent validation claim.

[8] Chacon, S., and Straub, B. Pro Git. Second edition. Apress, 2014. https://git-scm.com/book/en/v2

[9] PostgreSQL Global Development Group. PostgreSQL Documentation: Constraints. https://www.postgresql.org/docs/current/ddl-constraints.html

[10] Doyle, J. “A Truth Maintenance System.” Artificial Intelligence 12(3), 1979, pp. 231–272. https://doi.org/10.1016/0004-3702(79)90008-0

[11] Moreau, L., and Missier, P., eds. PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-dm/

[12] Meyer, B. “Applying Design by Contract.” Computer 25(10), 1992, pp. 40–51. https://doi.org/10.1109/2.161279

[13] McPhetridge, M. D. MPLPB originality and priority reassessment. `docs/research/mplpb-originality/README.md` at the commit in [4]. General logical principles are distinguished from claims about the implementation's integration.